

บทที่ 4

Java Servlet , JSP

4.1 บทนำ

แม้ว่าโลกของอินเทอร์เน็ตจะเพิ่งเกิดขึ้นเพียงไม่กี่ปีก็ตาม แต่เทคโนโลยีที่ใช้กับอินเทอร์เน็ตกลับมีการเปลี่ยนแปลงไปอย่างรวดเร็วมาก ในสมัยแรกๆ หน้าเพจต่างๆ ที่อยู่ในเว็บจะเป็นลักษณะของสแตติกเพจ หรือเพจที่ไม่มีการเปลี่ยนแปลงเนื้อหาที่ไม่ว่าจะนานเท่าไร นอกเสียจากว่าผู้ดูแลเพจนั้นจะทำการอัปเดตเพจดังกล่าว เพจลักษณะนี้เป็นเพจที่นิยมใช้กันทั่วไปในอินเทอร์เน็ตสมัยแรกเพราะอินเทอร์เน็ตยังนิยมกันอยู่ในวงแคบ โดยกลุ่มผู้ใช้จะเป็นกลุ่มบุคคลที่อยู่ในวงการศึกษาเท่านั้น ต่อมาจากนั้นไม่นานทางผู้ผลิตบราวเซอร์ได้ทำการเพิ่มความสามารถให้กับเพจโดยอนุญาตให้เพจสามารถแทรกสคริปต์เล็กๆ ลงไปพร้อมกับส่วนที่เป็น HTML ได้ซึ่งจุดนี้ก็คือจุดเริ่มต้นของจาวาสคริปต์ทางฝั่งไคลเอนต์นั่นเอง

แม้ว่าเพจจะเริ่มมีความสามารถในการโต้ตอบกับผู้ใช้โดยอิงความสามารถจากจาวาสคริปต์แล้วก็ตาม ถ้าพูดถึงในแง่ของส่วนเนื้อหาของตัวเพจจริงๆ แล้วตัวเพจเองก็ยังคงเป็นเพจสแตติก อยู่เช่นเดิม เมื่อกลุ่มผู้ใช้อินเทอร์เน็ตเริ่มมีมากขึ้นความต้องการที่จะทำให้เพจสามารถทำการรับส่งข้อมูลรวมไปถึงการเปลี่ยนแปลงเนื้อหาได้โดยอัตโนมัติก็เกิดขึ้น เทคโนโลยีที่เกิดขึ้นเพื่อรองรับความต้องการเหล่านี้ก็คือแอปพลิเคชันทางฝั่งเซิร์ฟเวอร์นั่นเอง

Server Side Application ในระยะแรก ๆ มักถูกเขียนขึ้นด้วยคอนเซ็ปต์ของ CGI (Common Gateway Interface) โดยหลักการทำงานง่าย ๆ ก็คือเว็บเบราว์เซอร์จะทำการส่งข้อมูลที่เกิดจาก Action ของ User เช่น การคลิกลิงค์หรือการกรอกแบบสอบถามไปยังเว็บเซิร์ฟเวอร์โดยแทนที่เว็บเซิร์ฟเวอร์จะทำการส่งเพจที่เป็น static page กลับมาเว็บเซิร์ฟเวอร์จะทำการ forward เด้าดังกล่าวไปยังโปรแกรมซึ่งถูกจัดไว้ โปรแกรมดังกล่าวจะทำการประมวลผลเด้าที่ได้แล้วจะส่งผลกลับไปยังเว็บเซิร์ฟเวอร์ซึ่งทางเว็บเซิร์ฟเวอร์ก็จะส่งผลที่ได้นี้กลับไปยังเว็บเบราว์เซอร์อีกทีหนึ่ง

4.2 Servlet

เซิร์ฟเลต คือเทคโนโลยี ที่เกิดขึ้นมาจาก Java เพื่อใช้งานทางด้านการสร้าง โปรแกรมในรูปแบบ CGI สำหรับทำงานบนเว็บเซิร์ฟเวอร์ เพื่อให้ผลการแสดงเว็บสู่ผู้อ่าน มีการเปลี่ยนแปลงตามการใช้งาน (On The Fly) ซึ่งเป็นเทคโนโลยีที่สืบเนื่องมาจากเหตุผลดังนี้

ความต้องการให้หน้าเว็บเปลี่ยนตามข้อมูลที่ผู้อ่านส่งให้ เช่น เว็บที่ให้บริการระบบค้นหา (Search Engine)ทั้งหลาย ที่ได้รับข้อมูลจากผู้ใช้ คือข้อความที่ต้องการค้นหา และแสดงผลลัพธ์หน้าเว็บ ตามข้อมูลที่ค้นหาเจอแก่ผู้ใช้

ความต้องการให้หน้าเว็บเปลี่ยนตามข้อมูลที่มีการเปลี่ยนแปลงบ่อยครั้ง เช่น เว็บที่เกี่ยวข้องกับการรายงานข่าว หรือสภาพอากาศ จะเห็นว่าข้อมูลที่รายงานสู่ผู้ใช้ ดังกล่าว มีการเปลี่ยนแปลงอยู่ตลอดเวลา อาจเป็นรายชั่วโมง หรือต่อนาทีก็ได้

ความต้องการให้หน้าเว็บเปลี่ยนตามระบบฐานข้อมูล เช่น ระบบควบคุมคลังสินค้า ที่อาศัยการเก็บข้อมูลในระบบฐานข้อมูล สามารถแสดงผลจำนวนสินค้า ตามข้อมูลที่มีอยู่จริงในฐานข้อมูล ออกสู่ผู้ใช้บนระบบเว็บ

ข้อแตกต่างของ Servlet เมื่อเทียบกับ CGI

- เซิร์ฟเลต เป็นเทคโนโลยี ที่มีรูปแบบการทำงานเหมือนกับ เทคโนโลยี CGI ที่มีการใช้งานอยู่ แต่เนื่องจาก เซิร์ฟเลต ถูกพัฒนามาจากเทคโนโลยีจาวาอีกที ดังนั้น ต่อไปนี้คือข้อเปรียบเทียบซึ่งแสดงให้เห็น ถึงความแตกต่างระหว่าง เซิร์ฟเลต และ CGI

- ประสิทธิภาพ การทำงานของ CGI จะมีการสร้างโปรเซส (Process) ใหม่ทุกครั้ง ที่มีการเรียกใช้งานจากผู้ใช้งาน ดังนั้นหากมีการเรียกใช้งาน จำนวนหลายครั้งในช่วงเวลาเดียวกัน ก็จะมีจำนวนโปรเซสเกิดขึ้น เท่ากับจำนวนครั้งที่เรียกใช้งาน โปรเซสจะมีขนาดและรูปแบบการทำงาน ที่เหมือนกัน ทำให้กินเวลาในการทำงานตามจำนวนโปรเซส แต่สำหรับ เซิร์ฟเลต แล้ว ระบบเวอร์ชวลแมชีน (Java Virtual Machine) จะถูกดึงขึ้นมาทำงานก่อน เพื่อทำให้เกิดโปรเซสตามลักษณะการเรียกใช้ของผู้ใช้เช่นกัน แต่การทำงานของเซิร์ฟเลต จะเป็นรูปแบบแธรด(Thread) แทนที่จะเรียกว่าโปรเซส ซึ่งแต่ละแธรดขนาดการทำงานเล็กกว่าเดิม และกินระยะเวลาการทำงานน้อยกว่า จึงทำให้รูปแบบของ เซิร์ฟเลต มีประสิทธิภาพกว่านั่นเอง

- ความสะดวก หากคุณสามารถเขียนโปรแกรม ด้วยภาษา C, C++ หรือ Java อยู่แล้ว จะเกิดความสะดวกและรวดเร็วในการสร้าง เซิร์ฟเลต มากกว่า แทนที่จะเริ่มศึกษาภาษาคำสั่งใหม่ๆ และ เซิร์ฟเลต ก็มีความสามารถเพียงพอสำหรับการใช้งานบนเว็บอยู่แล้ว เช่น การรับข้อมูลจากฟอร์ม, การอ่านและตั้งค่า HTTP Header, การทำงานกับ Cookie, การทำงานแบบ Session และอื่นๆอีกมาก

- ความสามารถ เซิร์ฟเลต มีข้อดีที่เห็นได้ชัดคือ สามารถติดต่อกับเว็บเซิร์ฟเวอร์ได้โดยตรง ในขณะที่ CGI ปกติทำไม่ได้ (ยกเว้นรุ่นใหม่ๆ) และ เซิร์ฟเลต ยังสามารถแชร์ข้อมูลการทำงาน ร่วมกับโปรแกรมอื่นๆได้ด้วย

- การโยกย้าย เนื่องจาก เซิร์ฟเลต ถูกสร้างมาจาก Java ซึ่งเป็นเทคโนโลยี ที่สามารถถูกใช้งาน ณ. ที่ใดก็ได้ โดยไม่จำกัดรูปแบบ ดังนั้น หากพัฒนา เซิร์ฟเลต เพื่อใช้งานแล้ว สามารถที่จะโยกย้ายผลผลิตไปยังระบบอื่นๆ ได้ โดยไม่ต้องปรับปรุงหรือเปลี่ยนแปลง
- ไม่แพง ในปัจจุบันเว็บเซิร์ฟเวอร์ที่สนับสนุนการทำงาน เซิร์ฟเลต มีอยู่มากมาย ทั้งที่เป็นแบบฟรี (ดาวน์โหลด) และซื้อ-ขายกัน นอกจากนี้ ในอนาคตแผนการผลิตเว็บเซิร์ฟเวอร์อื่นๆ ยังมีการสนับสนุนการใช้งาน เซิร์ฟเลต อีกด้วย

4.3 JSP (Java Server Pages)

JSP(Java Server Pages) หรือที่เรียกกันง่ายๆว่า JSP เป็นเป็นภาษาที่ใช้ในการเขียนเซิร์ฟเลตซึ่งเป็น web-scripting เทคโนโลยีคล้ายกับ Netscape server-side JavaScript (SSJS) หรือ Microsoft Active Server Pages (ASP) แต่ผิดกันตรงที่หัวใจหลักของ JSP คือภาษาจาวาซึ่งเป็นภาษาที่แนวคิดหลักอยู่ที่เชิงวัตถุ (object-oriented style) ซึ่งช่วยทำให้ง่ายต่อการพัฒนาในโปรเจกต์ใหญ่ ๆ ตลอดจนสามารถนำส่วนประกอบต่าง ๆ กลับมาใช้ได้อีก (software reusable) และเป็นเทคโนโลยีที่เป็นผลรวม ของการนำเอาหลักการสร้างเว็บแบบสแตติกด้วย HTML มารวมกับการสร้างเว็บแบบไดนามิก ก่อให้เกิดการแสดงผลเว็บ ที่มีการเปลี่ยนแปลงข้อมูลได้ตามการใช้งานของผู้ใช้ เห็นได้ว่าหากพูดถึงหลักการนำเสนอเว็บแล้ว เป็นหลักการที่ทำให้เว็บมีการเปลี่ยนแปลงข้อมูล เช่นเดียวกับ ที่ใช้โปรแกรมจำพวก CGI ทั่วไป แต่ในความหมายของ JSP แล้ว เป็นขั้นตอนและวิธีการสร้างที่ทำงานร่วมกับคำสั่ง HTML และเก็บบันทึกไว้ในไฟล์เดียวกัน แต่ในขณะที่ CGI เป็นโปรแกรมที่ผลิต HTML ให้เมื่อถูกสั่งให้ทำงาน

ข้อแตกต่างของ JSP เมื่อเทียบกับเทคโนโลยีอื่นๆ

- JSP ไม่ได้เป็นเทคโนโลยีเดียว ในปัจจุบันที่สามารถทำให้เว็บ แสดงข้อมูลในแบบไดนามิกได้ มีเทคโนโลยีอื่นๆที่ทำงานในลักษณะนี้หลายแบบ เนื้อหาส่วนนี้ เป็นการแนะนำให้ผู้อ่านได้เห็นข้อแตกต่างบางประการ เมื่อเทียบกับ JSP
- Active Server Page(ASP) เป็นเทคโนโลยีที่เหมือนกับ JSP แต่เป็นเทคโนโลยีที่พัฒนามาจากบริษัทไมโครซอฟต์ ผู้ผลิตระบบปฏิบัติการวินโดวส์ที่รู้จักกัน โดยทั่วไป ลักษณะ JSP มีรูปแบบที่แตกต่างกันเด่นชัด 2 ประการคือ ประการแรก JSP สามารถสร้างได้จาก ภาษาจาวา ต่างจากภาษา VB หรือภาษาใดๆ ซึ่งเป็นข้อกำหนดเฉพาะของ ไมโครซอฟต์ ประการที่สอง JSP ซึ่งเป็นผลพวงมาจากประการแรกคือ สามารถโยกย้ายการทำงาน ไปใช้งานระบบปฏิบัติการใดๆก็ได้ ที่มีการใช้งานกันอยู่ในปัจจุบัน โดยไม่ต้องแก้ไขหรือเปลี่ยนแปลงใดๆ ในขณะที่ ASP ก็โยกย้ายได้เช่นเดียวกัน แต่ต้องอยู่ในระบบปฏิบัติการที่ ไมโครซอฟต์ กำหนดขึ้นเท่านั้น

- JSP สามารถทำงานได้เช่นเดียวกับที่ เซิร์ฟเลต ทำได้ แต่มีจุดเด่นมากกว่าที่ JSP มีความสะดวกในการสร้าง และเปลี่ยนแปลง มากกว่า เพราะทำโดยตรงที่ไฟล์ HTML มากกว่าการที่ต้องลงมือเขียนคำสั่งภาษาจาวา โดยตรงเหมือนกันสร้าง เซิร์ฟเลต ซึ่งต้องมีการนำไปคอมไพล์ ก่อนการนำไปใช้งาน

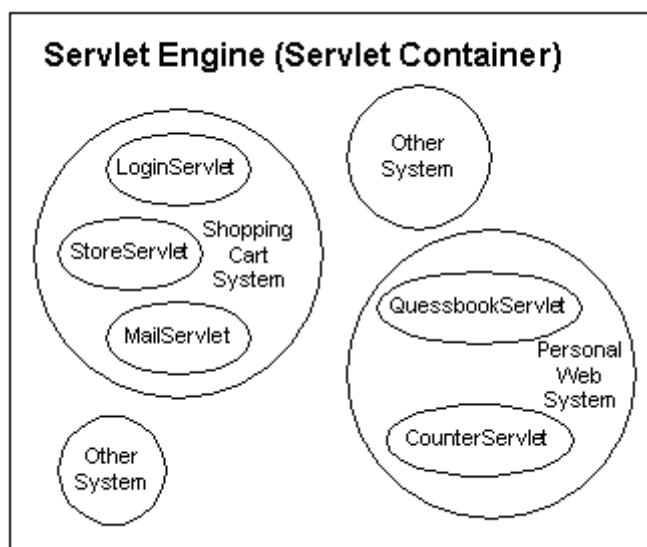
- Server-Side Includes (SSI) SSI คือรูปแบบการสร้างไดนามิกเว็บเช่นเดียวกัน แต่รูปแบบการทำงานคือ การนำเอาข้อมูลที่มีอยู่แล้ว บนเซิร์ฟเวอร์นำมาประกอบใส่ ในเว็บเท่านั้น ต่างจาก JSP ซึ่งสามารถนำข้อมูลมาใส่ แต่มีรูปแบบในการทำงาน มีรูปแบบในการประมวลผล หรือการเรียกโปรแกรมภายนอก เช่น เซิร์ฟเลต มาช่วยในการทำให้ข้อมูล มีรูปแบบต่างๆ เช่น ดึงข้อมูลมาจากระบบฐานข้อมูล (Database) นอกจากข้อมูลที่มีอยู่บนเซิร์ฟเวอร์เช่นที่ใช้ใน SSI เท่านั้น

- JavaScript เป็นการทำเนื้อหาไดนามิกให้กับเว็บ แต่รูปแบบการทำงานเกิดขึ้นของ JavaScript เกิดจากการประมวลผลและดึงข้อมูล ที่มีอยู่บนเครื่องผู้ใช้ หรือไคลเอนท์ เท่านั้น มารวบรวมเนื้อหาในเว็บ ต่างกับ JSP ที่เป็นการทำไดนามิก แต่ข้อมูลถูกสร้าง และดึงมาจากระบบเซิร์ฟเวอร์ ที่มีหลากหลายมากกว่า และทำให้เนื้อหาข้อมูลเป็นเนื้อเดียวกัน เมื่อผู้ใช้เรียกดู แต่ JavaScript เนื้อข้อมูลเป็นของเครื่องผู้ใช้งาน ซึ่งผู้ใช้งานคน (ต่างเครื่อง) ก็จะให้ข้อมูลที่คล้ายกัน คือไม่เหมือนกันทั้งหมด

- สเตติก HTML (Static HTML) เน้นอนชื่อนี้ คงเห็นได้ชัดเจนที่ว่า สเตติก HTML ให้เนื้อหาข้อมูลที่คงที่ตลอดเวลา ไม่ว่าจะถูกเรียกดูในเวลาใดๆ (ยกเว้นมีคนมาแก้ไขคำสั่ง HTML) ในขณะที่ JSP ทำให้เนื้อหาข้อมูลมีการเปลี่ยนแปลงตามการใช้งาน หรือเครื่องผู้ใช้ โดยไม่ต้องเปลี่ยนแปลงแก้ไขคำสั่ง JSP

4.4 เซิร์ฟเลตเอนจิน (Servlet Engine)

ในการรันระบบที่เขียนขึ้นโดยใช้หลักการของเซิร์ฟเลตเราจะต้องนำระบบดังกล่าวมาบรรจุอยู่ในสิ่งๆ หนึ่งที่เรียกว่าเซิร์ฟเลตเอนจิน ให้นึกว่าเซิร์ฟเลตเอนจินคล้ายๆกับกล่องๆหนึ่งที่ใส่ลูกปิงปองไว้หลายลูก โดยลูกปิงปองแต่ละลูกก็คือระบบๆ หนึ่งนั่นเอง หลายคนอาจสงสัยทำไมถึงใช้คำว่าระบบ โดยทั่วไปแอปพลิเคชันทางฝั่งเซิร์ฟเวอร์หนึ่งๆ ที่ถูกเขียนขึ้นโดยใช้ API ของเซิร์ฟเลตจะถูกเรียกว่าเซิร์ฟเลต ในหนึ่งระบบอาจประกอบด้วยเซิร์ฟเลตหลายอัน ยกตัวอย่างเช่น ระบบที่เกี่ยวกับ Shopping Cart อาจจะประกอบด้วยเซิร์ฟเลตที่ทำหน้าที่ในการเช็คสต็อกอิน, เซิร์ฟเลตที่ทำหน้าที่ในการเก็บข้อมูลสินค้า, เซิร์ฟเลตที่ทำหน้าที่ในการส่งเมลล์กลับไปยังลูกค้าเพื่อบอกว่าได้ทำการส่งของไปให้แล้ว เป็นต้น ดังนั้นถ้ามองโดยรวมแล้วเซิร์ฟเลตเอนจินก็คือที่รวมของระบบตั้งแต่หนึ่งระบบถึงหลายระบบ โดยแต่ละระบบจะประกอบด้วยเซิร์ฟเลตหนึ่งอันหรือมากกว่า ดังรูปที่ 4-1 ซึ่งระบบในที่นี้อาจจะหมายถึง Zone (Apache Jserv) หรือ Web Application (Tomcat) ก็ได้



รูปที่ 4-1 เซิร์ฟเลตเอนจิน และเซิร์ฟเลตที่อยู่ภายใน

เซิร์ฟเลตเอนจินเป็นเพียงกล่องๆหนึ่งที่ใช้บรรจุและรันกลุ่มของเซิร์ฟเลตเท่านั้น ในการที่จะทำการติดต่อสื่อสารกับไคลเอนต์ ตัวเซิร์ฟเลตเอนจินนี้จะต้องทำงานร่วมกับเว็บเซิร์ฟเวอร์ ซึ่งเปรียบเสมือนจากหน้าที่ติดต่อกับไคลเอนต์อีกทีหนึ่ง เมื่อใดก็ตามที่มีคำร้องขอส่งมาจากไคลเอนต์ ถ้าคำร้องขอนั้นจะจงมาที่ตัวเซิร์ฟเลต ทางเว็บเซิร์ฟเวอร์ก็จะทำการส่งคำร้องขอนั้นมาให้เซิร์ฟเลตเอนจิน ซึ่งทางเซิร์ฟเลตเอนจินก็จะทำการเรียกเซิร์ฟเลตที่ไคลเอนต์ต้องการขึ้นมาทำการประมวลผลคำร้องขอนั้น โดยท้ายสุดเซิร์ฟเลตจะส่งผลกลับไปให้เซิร์ฟเลตเอนจิน เซิร์ฟเลตเอนจินก็จะส่งผลที่ได้กลับไปให้เว็บเซิร์ฟเวอร์ ซึ่งเว็บเซิร์ฟเวอร์ก็จะส่งผลกลับไปให้ไคลเอนต์

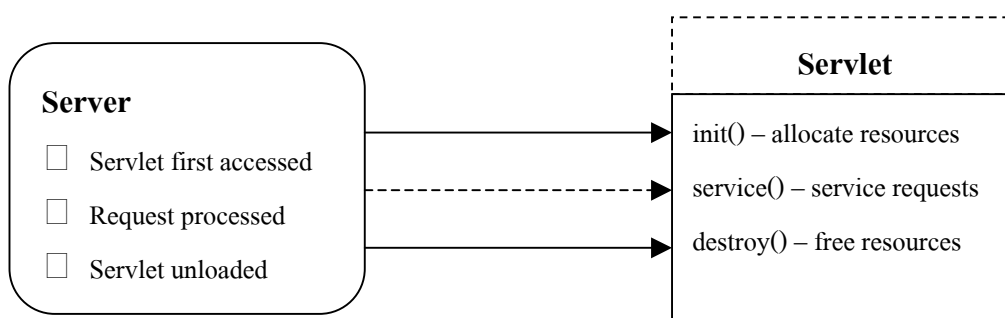
เซิร์ฟเลตเอนจินอาจจะเป็นส่วนที่ติดมากับเว็บเซิร์ฟเวอร์อยู่แล้วยกตัวอย่างเช่น เซิร์ฟเลตเอนจิน ที่อยู่ในเซิร์ฟเวอร์ของ Netscape Enterprise, IBM Web Sphere หรืออาจจะเป็นส่วนที่เป็นส่วนที่เพิ่มเติมให้กับเว็บเซิร์ฟเวอร์ก็ได้เช่น Apache Jserv, Tomcat, JRun หรือแม้กระทั่งเป็นส่วนหนึ่งที่อยู่ในเว็บแอปพลิเคชัน เช่น BEA Web logic เป็นต้น ทั้งนี้การเลือกใช้เซิร์ฟเลตเอนจินแต่ละชนิดก็มักขึ้นอยู่กับปัจจัยหลายอย่างเช่น ความสะดวกในการรวมระบบที่จะสร้างขึ้นมาใหม่กับระบบที่มีอยู่แล้ว, งบประมาณที่มีอยู่สำหรับโครงการหรืออาจจะรวมไปถึงทักษะและประสบการณ์ส่วนตัวของนักพัฒนาแต่ละคน

4.5 อินเทอร์เน็ต javax.servlet.Servlet

หัวใจของเซิร์ฟเลตจริงๆอยู่ที่อินเทอร์เน็ตที่ชื่อ javax.servlet.Servlet โดยทุกเซิร์ฟเลตที่ถูกเขียนขึ้นจะต้องทำการอิมพลิเมนต์ตัวอินเทอร์เน็ตนี้ไม่ทางตรงก็ทางอ้อม (ทางตรงก็คือการอิมพลิเมนต์ตัวอินเทอร์เน็ตเลย ส่วนทางอ้อมก็คือการให้เซิร์ฟเลตทำการแบ่งย่อยคลาสบางคลาสที่ได้ทำการอิมพลิเมนต์ตัวอินเทอร์เน็ตไว้แล้ว) เหตุผลว่าทำไมเราต้องอิมพลิเมนต์ตัวอินเทอร์เน็ตนี้เพราะว่าเมื่อไรก็ตามที่มีคำร้องขอจากไคลเอนต์เข้ามายังเซิร์ฟเลตเอนจิน ตัวเซิร์ฟเลตเอนจินจะทำการหาเซิร์ฟเลตที่การร้องขอดังกล่าวอ้างถึง หลังจากนั้นเซิร์ฟเลตเอนจินจะทำการเรียกฟังก์ชันต่างๆที่อยู่ในเซิร์ฟเลตเพื่อทำการประมวลผลคำร้องขอของไคลเอนต์ โดยฟังก์ชันที่เซิร์ฟเลตเอนจินจะทำการเรียกก็คือฟังก์ชันที่เซิร์ฟเลตได้ทำการอิมพลิเมนต์ ซึ่งเป็นฟังก์ชันที่ถูกกำหนดอยู่ใน javax.servlet.Servlet อินเทอร์เน็ตนั่นเอง อาจมีคำถามว่าทำไมเซิร์ฟเลตเอนจินถึงเรียกฟังก์ชันที่กำหนดอยู่ในอินเทอร์เน็ตนี้ เหตุผลง่ายๆก็คือเซิร์ฟเลตเป็นส่วนที่ถูกโหลดเข้าไปในเซิร์ฟเลตเอนจินในช่วง Runtime ตัวเซิร์ฟเลตเอนจินเองไม่สามารถทราบได้ว่าเซิร์ฟเลตต่างๆมีฟังก์ชันอะไรประกอบอยู่บ้างนอกเสียจากว่าเซิร์ฟเลตนั้นได้ทำการอิมพลิเมนต์ฟังก์ชันที่เป็นมาตรฐานที่เซิร์ฟเลตเอนจินรับรู้ ซึ่งนี่ก็คือเหตุผลว่าทำไมทุกเซิร์ฟเลตจะต้องทำการอิมพลิเมนต์ตัว javax.servlet.Servlet อินเทอร์เน็ต อย่างไรก็ตามเราสามารถให้เซิร์ฟเลตเอนจินเรียกฟังก์ชันอื่นๆที่อยู่ในเซิร์ฟเลตได้ ซึ่งวิธีการก็คือการใส่ฟังก์ชันดังกล่าวเข้าไปในส่วนอิมพลิเมนต์ของฟังก์ชันต่างๆที่ถูกกำหนดอยู่ใน javax.servletServlet อินเทอร์เน็ต

4.6 วงจรชีวิตของเซิร์ฟเลต (Servlet's Life Cycle)

วงจรชีวิตของเซิร์ฟเลตจะประกอบด้วย 3 สถานะคือ สถานะเริ่มต้น (init()), สถานะให้บริการ(service()) และสถานะทำลาย (destroy()) เมธอดเหล่านี้เป็น “Callback method” หมายถึงเมธอดจะไม่ถูกเรียกโดยตัวเซิร์ฟเลตเอง แต่จะถูกเรียกโดยเซิร์ฟเลตเอนจิน ในการตอบสนองกับเหตุการณ์ (event) ที่เกิดขึ้น



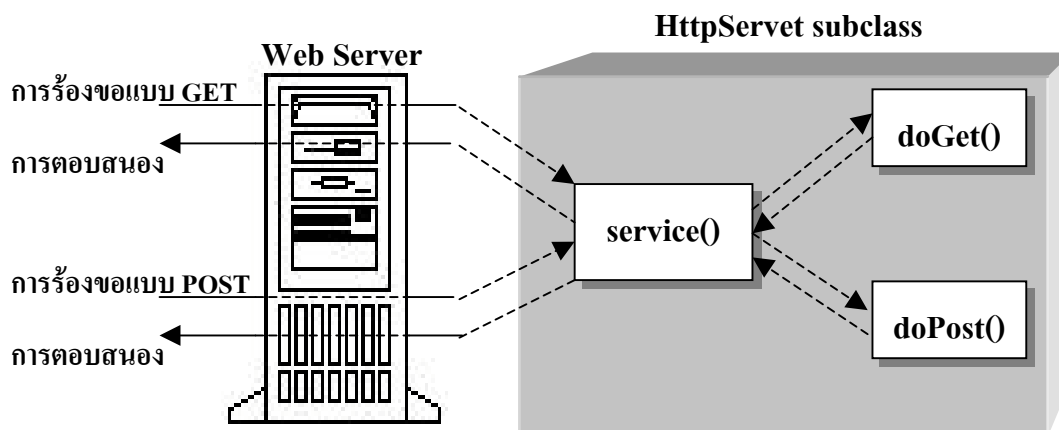
รูปที่ 4-2 วงจรชีวิตของเซิร์ฟเลต

4.6.1 สถานะเริ่มต้น (Initial State)

เป็นการใช้เมธอด `init()` ซึ่งเมธอดจะถูกเรียกโดยเซิร์ฟเวอร์ทันทีที่อินสแตนซ์ (instant) ถูกสร้างเสร็จเรียบร้อยแล้ว โดยปกติเมธอดนี้จะถูกใช้เพื่อทำการเริ่มต้นเซิร์ฟเลต สาเหตุที่ไม่ใช้คอนสตรัคเตอร์ (constructor) เพราะคอนสตรัคเตอร์ของ JDK 1.0 (เริ่มมีเซิร์ฟเลต) ไม่สามารถรับอาร์กิวเมนต์ (argument) ในกรณีที่คลาสถูกโหลดแบบไดนามิก (dynamic) ได้ ดังนั้นการที่จะจัดหาข้อมูลเกี่ยวกับตัวมันเองและสิ่งแวดล้อมให้แก่เซิร์ฟเลตที่ถูกโหลดเป็นครั้งแรก เซิร์ฟเวอร์จำเป็นต้องเรียกเมธอด `init()` และส่งผ่านออบเจกต์ (object) ที่ใช้อินเตอร์เฟซ `ServletConfig` นอกจากนี้ภาษาจาวาไม่อนุญาตให้อินเตอร์เฟซมีการประกาศคอนสตรัคเตอร์ นั่นหมายถึงอินเตอร์เฟซของแพ็คเกจ (package) `javax.servlet.Servlet` ก็ไม่สามารถประกาศคอนสตรัคเตอร์ที่รับค่าพารามิเตอร์ (parameter) `ServletConfig` ได้เช่นกัน จึงต้องประกาศเมธอดชื่ออื่นแทนเช่น `init()` แต่ยังคงมีทางเป็นไปได้ที่จะประกาศคอนสตรัคเตอร์ให้แก่เซิร์ฟเลต แต่ในคอนสตรัคเตอร์ดังกล่าวห้ามมีการเข้าถึงออบเจกต์ `ServletConfig` และไม่สามารถ throw `ServletException` ได้ ออบเจกต์ `ServletConfig` จะส่งพารามิเตอร์ที่จำเป็นต่อการเริ่มต้นการทำงานให้แก่เซิร์ฟเลต พารามิเตอร์ดังกล่าวจะถูกป้อนให้แก่เซิร์ฟเลต โดยไม่เกี่ยวข้องกับการร้องขอที่เข้ามา

4.6.2 สถานะให้บริการ (Service State)

สถานะให้บริการจะถูกเรียกจากเซิร์ฟเวอร์ทุกครั้งที่เกิดเอนต์ทำการร้องขอใช้บริการเซิร์ฟเลต เมธอด `service()` จะรับค่าพารามิเตอร์ 2 ค่าได้แก่ออบเจกต์การร้องขอและออบเจกต์การตอบสนอง ออบเจกต์การร้องขอจะบอกเซิร์ฟเลตเกี่ยวกับการขอใช้บริการที่มีเข้ามา ในขณะที่ออบเจกต์การตอบสนองจะถูกเรียกใช้ในการส่งการตอบรับคืนกลับให้แก่เซิร์ฟเวอร์ โดยปกติแล้วเมธอด `service()` จะไม่ถูกโอเวอร์ไรด์ (override) แต่จะทำการโอเวอร์ไรด์เมธอด `doGet()` เพื่อใช้จัดการการร้องขอแบบ GET และเมธอด `doPost` สำหรับจัดการกับการร้องขอแบบ POST แทน ดังตัวอย่างในรูป



รูปที่ 4-3 วิธีการจัดการกับร้องขอแบบ GET และ POST ของเซิร์ฟเลต

4.6.3 สถานะทำลาย (Destroy State)

สถานะทำลายเซิร์ฟเวอร์จะเรียกเมธอด `destroy()` ของเซิร์ฟเลต เมื่อเซิร์ฟเลตกำลังจะถูกอันโหลด (unload) ในเมธอด `destroy()` ควรมีการคืนทรัพยากรต่างๆกลับคืนมาให้แก่ระบบ เพื่อที่โปรแกรมอื่นๆจะได้นำทรัพยากรเหล่านี้ไปใช้ได้ นอกจากนี้เมธอดยังให้โอกาสเซิร์ฟเลตในการบันทึกสถานะของข้อมูลต่างๆที่ถูกเรียกใช้เป็นประจำจากการเรียกเมธอด `init()` ในครั้งต่อไป

4.7 การติดต่อฐานข้อมูลด้วย JDBC

การทำงานกับฐานข้อมูลจะใช้สถาปัตยกรรมที่เรียกว่า 3-Tier โดยมี Tier ที่ 1 คือ client Tier ที่ 2 คือ เซิร์ฟเวอร์ที่เชื่อมแอปพลิเคชันไปยัง Tier แรก และ Tier ที่ 3 คือเซิร์ฟเวอร์ที่เก็บข้อมูลในระบบไฟล์ หรือ ฐานข้อมูล ซึ่งการติดต่อไปยัง Tier ที่ 3 ที่เป็น Relational Database โดยจะต้องใช้อินเตอร์เฟซที่เรียกว่า JDBC (Java Database Connectivity)

JDBC ซึ่งเป็นมิดเดิลแวร์ตัวหนึ่ง มีรูปแบบและชื่อเรียกที่แบ่งออกเป็น 4 แบบด้วยกันคือ Type 1, Type 2, Type 3 และ Type 4 โดยเป็นข้อกำหนดที่ถูกกำหนดมาโดยบริษัทซัน ไมโครซิสเต็มส์ ซึ่งเป็นผู้กำหนดรายละเอียดของแต่ละประเภท ดังนี้

4.7.1 ประเภท 1 (Type1: JDBC-ODBC Bridge)

เรียกกันสั้นๆว่า JDBC-ODBC บริดจ์ หมายถึง JDBC ที่ทำงานอยู่บนชั้นการสื่อสารมิดเดิลแวร์ที่ชื่อ ODBC อีกทีหนึ่ง เนื่องจากการทำงานของคอมพิวเตอร์บนพื้นฐานของระบบปฏิบัติการไมโครซอฟต์ วินโดวส์ มีมิดเดิลแวร์ที่ชื่อ ODBC เป็นมาตรฐานในการสื่อสารอยู่แล้ว และเนื่องมาจากระบบที่พัฒนาให้

ใช้งานบนวินโดวส์ส่วนใหญ่ก็มักจะถูกพัฒนาโดยอาศัย ODBC มาช่วยในการทำงานอยู่ด้วย ดังนั้นในการสร้างโปรแกรมด้วยภาษาจาวาให้สามารถทำงานอยู่บนพื้นฐาน ODBC เดิม เพื่อหลีกเลี่ยงปัญหาอันเกิดจากความยุ่งยากในการติดตั้ง และต้องแปลงระบบที่ใช้อยู่ไปเป็นระบบใหม่ทั้งหมด

ลักษณะของการนำไปใช้งาน

- ถูกใช้สำหรับการศึกษาวิธีการเขียนโปรแกรมจาวา เพื่อติดต่อกับ JDBC
- เหมาะกับระบบงานที่มี ODBC ในการทำงานอยู่แล้ว
- สำหรับระบบงานที่ทั้งหมดทำงานอยู่บนพื้นฐานของแพลตฟอร์มวินโดวส์

ข้อด้อยของการนำไปใช้งาน

- ไม่เหมาะกับการนำไปใช้งานบนระบบงานที่มีขนาดใหญ่ๆ เนื่องจากทำให้เกิดความซ้ำในการทำงาน และประสิทธิภาพในการทำงานที่ไม่ดี
- มีข้อมูลในส่วนโอเวอร์เฮด(Overhead) สูง เนื่องจากต้องมีส่วนในการติดต่อระหว่าง JDBC และ ODBC เพิ่มเติม
- ไม่สนับสนุนความสามารถทั้งหมดของมาตรฐาน JDBC เนื่องจากข้อจำกัดของ ODBC ที่มีรูปแบบการทำงานน้อยกว่า JDBC

4.7.2 ประเภท 2 (Type2: Partial Java Driver)

สำหรับ JDBC ประเภทนี้ ตัวไดรเวอร์ (มีดเคิลแวร์ของแต่ละผลิตภัณฑ์มักจะใช้คำว่าไดรเวอร์แทนมีดเคิลแวร์ตัวนั้นๆ) จะถูกโหลดไว้ที่ไคลเอนท์หรือที่เียร์ส่วนติดต่อกับผู้ใช้ ดังนั้นรูปแบบในการสร้างโปรแกรมไคลเอนท์จะต้องพยายามโหลดไดรเวอร์ของผลิตภัณฑ์เซิร์ฟเวอร์ที่ต้องการติดต่อด้วย เช่น ถ้ามีระบบฐานข้อมูล DB2 โปรแกรมในส่วนไคลเอนท์ต้องสร้างคำสั่งในการโหลดไดรเวอร์ DB2 ขึ้นใช้งาน

ลักษณะของการนำไปใช้งาน

- มีประสิทธิภาพในการดีกว่าประเภท 1 เมื่อเปรียบเทียบกัน
- คำสั่งในการติดต่อกับเซิร์ฟเวอร์ จะเป็นคำสั่งที่ดีที่สุดสำหรับเซิร์ฟเวอร์นั้นๆ โดยเฉพาะ ทำการทำงานโดยรวมดีกว่า

ข้อดีของการนำไปใช้งาน

- ผู้ใช้โปรแกรมในส่วนไคลเอนท์ยังต้องการไดรเวอร์ สำหรับเซิร์ฟเวอร์โดยเฉพาะ
- เมื่อมีการเปลี่ยนแปลงเซิร์ฟเวอร์เป็นผลิตภัณฑ์ตัวอื่น โปรแกรมในส่วนไคลเอนท์ต้องมีการเปลี่ยนแปลงและคอมไพล์ใหม่เสมอ

4.7.3 ประเภท 3 (Type 3: Pure Java Driver)

ในการใช้งานสถาปัตยกรรมแบบหลาย-tier ที่จำเป็นต้องมีแอปพลิเคชันเซิร์ฟเวอร์ทำงานร่วมอยู่ในระบบ ดังที่ผู้อ่านได้ศึกษามาแล้ว แอปพลิเคชันเซิร์ฟเวอร์จะมาลดข้อยุ่งยากในการติดตั้งหรือดูแลระบบในส่วนเซิร์ฟเวอร์ไคลเอนท์ ดังนั้นลักษณะของ JDBC ในประเภทที่ 3 คือการนำเอา JDBC ไดรเวอร์ไปวางไว้ที่ตัวแอปพลิเคชันเซิร์ฟเวอร์ เพื่อเป็นตัวเชื่อมโยงการสื่อสารไปหาเซิร์ฟเวอร์

ลักษณะของการนำไปใช้งาน

- เป็นระบบที่มีประสิทธิภาพในการทำงานที่ดีกว่าประเภทที่ 1 และ 2 เมื่อเปรียบเทียบกัน
- เหมาะสำหรับองค์กรที่มีเซิร์ฟเวอร์ทางด้านฐานข้อมูลที่หลากหลายรูปแบบ
- การทำงานของไคลเอนท์ไม่จำเป็นต้องติดตั้ง JDBC ไดรเวอร์ไว้ในทุกตัว
- การติดตั้งและการดูแลระบบไคลเอนท์ทำได้ง่าย และสะดวก

ข้อดีของการนำไปใช้งาน

- ยังต้องการไดรเวอร์สำหรับแต่ละผลิตภัณฑ์ เพื่อติดตั้งไว้ที่แอปพลิเคชันเซิร์ฟเวอร์

4.7.4 ประเภท 4 (Type 4: Direct-to-DB)

ลักษณะการทำงานของ JDBC ในประเภทที่ 4 มีการทำงานโดยการส่งคำสั่ง SQL จากไคลเอนท์ไปที่เซิร์ฟเวอร์ฐานข้อมูลโดยตรงผ่านระบบเครือข่าย โดยที่รูปแบบคำสั่ง SQL จะต้องเป็นไปตามข้อกำหนดของระบบฐานข้อมูลแต่ละตัวที่เลือกใช้อยู่ ดังนั้นจะเห็นว่าการโต้ตอบของเซิร์ฟเวอร์กับคำสั่ง SQL สามารถทำงานได้รวดเร็ว ทำให้ JDBC ประเภทนี้มีรูปแบบการทำงานที่รวดเร็ว แต่ก็จำเป็นต้องสร้างไคลเอนท์สำหรับฐานข้อมูลแต่ละระบบซึ่งในบางครั้งไคลเอนท์สำหรับฐานข้อมูลตัวหนึ่งจึงไม่สามารถนำไปใช้กับระบบฐานข้อมูลตัวอื่นๆได้

ลักษณะของการนำไปใช้งาน

- มีประสิทธิภาพในการทำงานดีที่สุด เมื่อเทียบกับทุกประเภทที่กล่าวมา

ข้อดีของการนำไปใช้งาน

- มีความยุ่งยากในการพัฒนาโปรแกรมเพื่อทำงานในฝั่งไคลเอนต์ เพราะผู้พัฒนาต้องเรียนรู้การทำงานของเซิร์ฟเวอร์แต่ละตัวก่อน

4.7.5 การใช้งาน JDBC

การเข้าใช้งานฐานข้อมูลด้วยภาษาจาวา มีรูปแบบอยู่ด้วยกันถึง 4 ชนิด การที่จะเลือกใช้วิธีใดก็ขึ้นอยู่กับความเหมาะสมของระบบว่าระบบถูกออกแบบมาให้มีลักษณะการทำงานแบบใด แต่ถึงแม้ว่าจะได้ข้อสรุปอย่างไรก็ตาม วิธีการลงมือเขียนโปรแกรมก็มีขั้นตอนที่คล้ายกัน ดังนี้

- ขั้นตอนเริ่มติดต่อกับฐานข้อมูล (Establish Database)
- ขั้นตอนการส่งคำสั่ง SQL ให้กับฐานข้อมูล (Send Query)
- ขั้นตอนการแสดงผลลัพธ์สู่ผู้ใช้ (Display Result)
- ขั้นตอนการปิดการติดต่อกับฐานข้อมูล (Close Connection)